

SAFEXPLAIN: a Complete Approach Towards Trustworthy AI-Based Safety-Critical Systems

Jaume Abella¹, Irune Agirre², Thanh Bui³, Frank Geujen⁴, Gabriele Giordana⁵, Carlo Donzella⁶,
Francisco J. Cazorla¹, Enrico Mezzetti¹, Axel Brando¹, Javier Fernandez², Irune Yarza², Joanes Plazaola²,
Robert Lowe³, Maria Ulan³, Rob Lavreysen⁴, Lucas Tosi⁴, Ilaria Bloise⁵, Lorenzo Feruglio⁵,
Ilaria Cinelli⁵, Stefano Lodico⁷, William Guarienti⁷, Giuseppe Nicosia⁷, Valeria Dallara⁷

¹ Barcelona Supercomputing Center, Spain

² Ikerlan Technology Research Centre, Basque Research and Technology Alliance (BRTA), Spain

³ RISE Research Institutes of Sweden, Sweden

⁴ Navinfo Europe, The Netherlands

⁵ AIKO s.r.l., Italy

⁶ Exida Development s.r.l., Italy

⁷ Exida Engineering s.r.l., Italy

Abstract—AI becomes increasingly important in safety-critical systems, especially in the case of autonomous systems, since navigation relies on AI for object detection and collision avoidance. However, safety-critical systems must adhere to functional safety standards that enforce software to be correct-by-construction, component decomposition to simplify design and validation, and the use of data only for testing purposes – not to design the system itself. AI in general, and Deep Learning (DL) in particular have opposed characteristics since they have error rates (e.g., due to mispredictions), AI/DL modules can only be designed and validated monolithically, and they build on data for their design (i.e. for training purposes). Hence, DL solutions are at odds with the development process of safety-critical systems.

A number of standards have recently emerged in different domains to reconcile the requirements of safety-critical systems with the characteristics of DL solutions, such as ISO 21448, ISO/IEC TR 5469, and ISO 8800, among others. However, there is a lack of realistic practice to design a DL-based safety-critical system in accordance with those regulations, and existing solutions only cover some aspects in isolation, and are often incompatible among them.

SAFEXPLAIN is a 3-year Horizon Europe project addressing this challenge. SAFEXPLAIN, which finishes in September 2025, has already reached its main goals providing specific and complementary solutions to all those challenges so that AI-based safety-critical systems can be designed, implemented and validated adhering to the relevant functional safety standards in domains such as automotive, space and railway. In particular, SAFEXPLAIN provides the concepts, processes, tools and frameworks addressing the challenge end-to-end, from concept to solution. This is proven by the successful application of the SAFEXPLAIN approach in three case studies from the automotive, space and railway domains, whose results will see the light very soon.

Index Terms—Safety, AI, DL, Development process, Validation

I. INTRODUCTION

Safety-critical systems, such as those in cars, planes, satellites, trains, and the like, increasingly rely on Artificial Intelligence (AI) for autonomous operation. However, all software implementing safety-related functionalities, or able to interfere with those, must undergo a development process to guarantee that safety-related hazards are avoided.

The development of safety-critical systems follows a well-known V-model, moving from safety goals to safety requirements, system architecture design, software and hardware architecture design, and implementation to obtain a

system that is intended to be safe by construction. Then, the testing phase takes place from unit testing up to full system testing against its safety requirements. The Functional Safety Management (FSM) defines the required systematic approach (e.g., steps, actions, technical considerations) for developing safety-critical systems and other lifecycle phases, from concept definition up to decommissioning and disposal (for a more detailed explanation we refer the reader to the IEC 61508 standard [1]).

In recent years, the capabilities of AI, and particularly Deep Learning (DL), to perform advanced functions, such as visual perception, have led to their adoption in safety-related systems like autonomous vehicles. Whenever these functionalities implement safety requirements, they are also subject to provide evidence of their adherence to Functional Safety standards such as IEC 61508 [1]. Thus, the DL subsystem that implements safety requirements must be compliant with applicable safety development and management processes [2]–[4]. However, the general DL-based systems development process clashes frontally with traditional safety development processes [1], [4], [5]. For example:

- 1) DL software is designed monolithically following empirical training processes with example training data, rather than implementing specific safety requirements.
- 2) DL software, as opposed to any other kind of software in safety-critical systems, cannot be considered as correct by design due to the data-driven nature and stochasticity in its engineering process.
- 3) DL software design is no longer independent of data, and its parameters are set empirically based on training datasets.
- 4) DL software imposes high-performance demands on the underlying hardware and its inherent complexity (both hardware and software) entails challenges to comply with safety standards. Moreover, there is a lack of guidance in the development process for safety-critical systems incorporating DL software.

Therefore, FSMs must be adapted to consider DL-based solutions. Along with FSMs, AI-based software architectures meeting the corresponding safety requirements must also be adapted, as part of the system architectural design – one of the steps of the development process. Finally, platform realizations and the required tools to implement the system must also be tailored accordingly to consider the specific characteristics of DL software in safety-related functional-

ties.

SAFEXPLAIN addresses the aforementioned challenges by means of an integral end-to-end approach and realization that includes the following elements:

- The extension of non-AI development processes with new steps particularly suited to consider AI software as part of the critical system. Such a development process is accompanied by all the documentation required in each step.
- Appropriate safety patterns for different DL usage levels (i.e. with diverse safety requirements) to enable the use of AI software in safety-critical systems. Those safety patterns include redundancy, supervision and appropriate decision functions, much in line with the safety culture.
- Platform support and tools allowing to realize those safety patterns for user applications on top of high-performance MultiProcessor Systems on Chip (MP-SoCs).

This paper sketches the main solutions devised for each such challenge, and the results achieved so far. In particular, the rest of the paper is organized as follows. Section II presents the AI Functional Safety Management describing the development process. Sections III presents the safety architecture patterns devised in the project. Section IV describes the AI-specific supervision methods considered. Section V introduces the SAFEXPLAIN platform. Section VI provides some information on the project case studies. Finally, Section VII summarizes the paper and the steps pending until the end of the project.

II. SAFEXPLAIN's FSM

The FSM defines a development strategy consisting of a set of procedures, guidelines, and templates that define how a project with functional safety considerations should be executed (planning, involved team, activities, documents, configuration management, modification procedures, etc.). The main goal of the FSM is to ease the definition, organization, and control of the information generated during safety-critical project development while fulfilling the requirements of functional safety standards.

In the system realization phase, safety-critical systems typically adhere to a V-model and are often organized into the following phases: System Concept Specification (Ph1), System Architecture Specification (Ph2), Module Detailed Design (Ph3), Implementation (Ph4), Module Testing (Ph5), Integrations testing (Ph6) and Validation Testing (Ph7). Traditional functional safety standards such as IEC 61508 and ISO 26262 [6] define separate V-model based procedures for safety-related software and hardware.

A. Artificial Intelligence Lifecycle

A key distinction in the DL development process, in contrast to that followed by traditional functional safety systems, lies in phases related to preparation of datasets and their utilization to create models by optimizing error functions derived from requirement specifications. Figure 1 illustrates a simplified DL lifecycle, as proposed by [2]. Following the definition of the requirements specification, the DL workflow progresses through data management, model training, and model verification steps, culminating in the deployment of the model on the target inference platform.

Data Management is one of the most labor-intensive and crucial processes in DL development and includes activities such as data collection and data preparation. During this

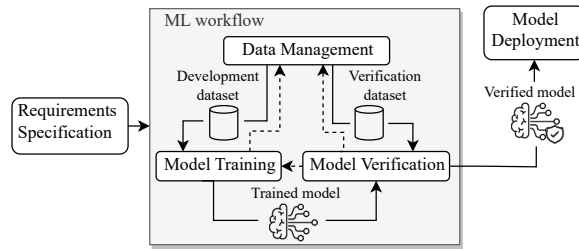


Fig. 1: Simplified DL Lifecycle [2]

process, the development dataset (comprising training and validation¹) and the verification dataset are generated. Subsequently, the model undergoes an iterative training process using the training dataset until the results on the validation dataset meet specific requirements in the model training step. Following successful training, the model verification evaluates the model's ability to properly extrapolate results with new data, utilizing the verification dataset. Lastly, the verified model is prepared in the model deployment phase for its implementation on the final inference platform. For an in-depth description guiding the AI lifecycle processes, we refer the reader to the ISO/IEC 5338 standard [7].

B. AI-FSM

The AI-FSM proposed in SAFEXPLAIN is grounded in state of the art practices from functional safety according to standards such as IEC 61508 and ISO 26262 as well as in emerging initiatives in the topic of AI safety, such as EASA Concept Paper [8], AMLAS [9], A-SPICE for Machine Learning (ML) [10] and ISO/IEC TR 5469 [4]. AI-FSM is publicly available at [11].

The V-based lifecycle, traditionally followed by FSM, has been expanded considering the peculiarities of the AI lifecycle, as depicted in Figure 2. Conventional lifecycle is represented by white boxes, while DL phases, along with those requiring slight modifications, are illustrated using colored boxes.

The traditional FSM defines procedures to be adhered to throughout the development of each task within the entire lifecycle, but it does not specifically address the nuances of the AI lifecycle [2]. The rest of this section delves into defining procedures relevant to DL tasks. The established phases of the traditional V-based development model are adopted as they are, while for the new processes/tasks, a neutral "linear" model is adopted. There is currently no reconciled way of straightforwardly adopting a V-model arrangement for such new processes/tasks, as witnessed e.g. by [8], [10].

The phases composing SAFEXPLAIN's AI-FSM are as follows (full details can be found in [12]):

- **Overall Lifecycle - Phase 0 (Ph0).** The Phase 0 (Ph0) is a transversal phase that collects all the generic project information, e.g., project document list, organizational chart, or tools selection. It must consider information collected in an AI-related safety project.
- **DL-Related Concept Specification - Phase 1 (Ph1).** Besides the traditional description of the use case and the definition of the operation reflected in the requirements, the use of AI involves the specific definition

¹Despite maintaining the term "validation" in line with AI nomenclature, it does not correspond to validation in the safety context.

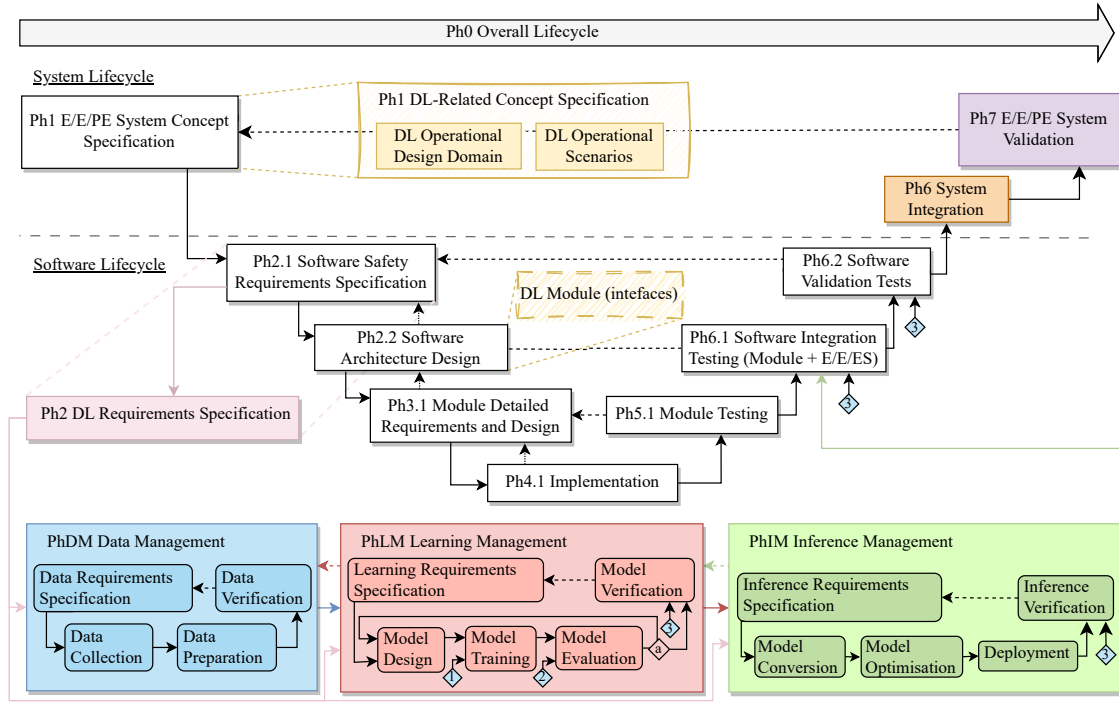


Fig. 2: Proposed AI Lifecycle

of the DL Operational Design Domain (ODD) and DL operational scenarios in which the DL will operate.

- **DL-Requirements Specification - Phase 2 (Ph2).** This phase entails the allocation of the software requirements specification to the DL component.
- **Data Management - Phase DM (PhDM).** The PhDM is responsible for ensuring the collection and preparation of data. This phase is crucial since the behavior of the DL components is determined by extracting patterns from data. Therefore, this phase entails the proper formulation of data requirements refined and allocated from DL requirements specification.
- **Learning Management - Phase LM (PhLM).** The Learning Management phase aims to optimize and get a DL model that meets the specified DL requirements. To achieve this, learning requirements must be specified, derived from DL requirements.
- **Inference Management - Phase IM (PhIM).** The purpose of this phase is to ensure that the target inference model still fulfills the specified DL requirements after adapting and even optimising the model for its deployment on the target hardware.

III. SAFEXPLAIN'S SAFETY ARCHITECTURE PATTERN

In this section, we provide a high-level description of a generic architectural pattern for safe AI execution, which can be tailored to different DL usage levels (omitted for space constraints). To this end, Figure 3 depicts a reference safety architecture pattern and this section describes its main safety elements, illustrated by means of enumerated rhombus symbols in the figure. These elements constitute the main building blocks to later define the specific safety architectural patterns for each DL usage level. Note that this safety patterns is inspired in the so-called E-Gas safety concept highly popular in the automotive domain [13], and used as

input in the definition of the main automotive safety standard (ISO 26262 [6]).

The goal of the reference safety architecture pattern is to show the main elements proposed in this section and to show an example on how they can be integrated together to build a safety critical system. This reference architecture shall be later tailored and adapted to the particular use case adopting it, and hence many different variants can be defined taking it as baseline.

In summary, the reference architecture pattern of Figure 3 integrates the following three safety mechanisms on it:

- 1) Diverse redundancy within the AI/ML constituent. Note that redundancy and diversity are commonly also applied to traditional systems, however we consider such part out of the scope of this paper, and we just focus on its application to the DL component.
- 2) Diagnostic and monitoring mechanisms. A hierarchical diagnostic and monitoring concept consisting of three levels is defined.
- 3) Supervision function. Its main goal is to check the appropriateness of the environment and to supervise the output of the ML constituent to identify unsafe situations and establish the limits for safe operation, providing a safe envelope.

All in all, the reference safety architecture pattern shown in Figure 3, consists of two main subsystems: the AI-based subsystem and the traditional subsystem, which we detail next.

A. AI-based subsystem

This subsystem comprises all the AI-related elements in the architecture.

- 1) **AI/ML constituent.** Consists of the ML model as well as the corresponding data pre-processing and post-processing components. As indicated before, the AI/ML constituent

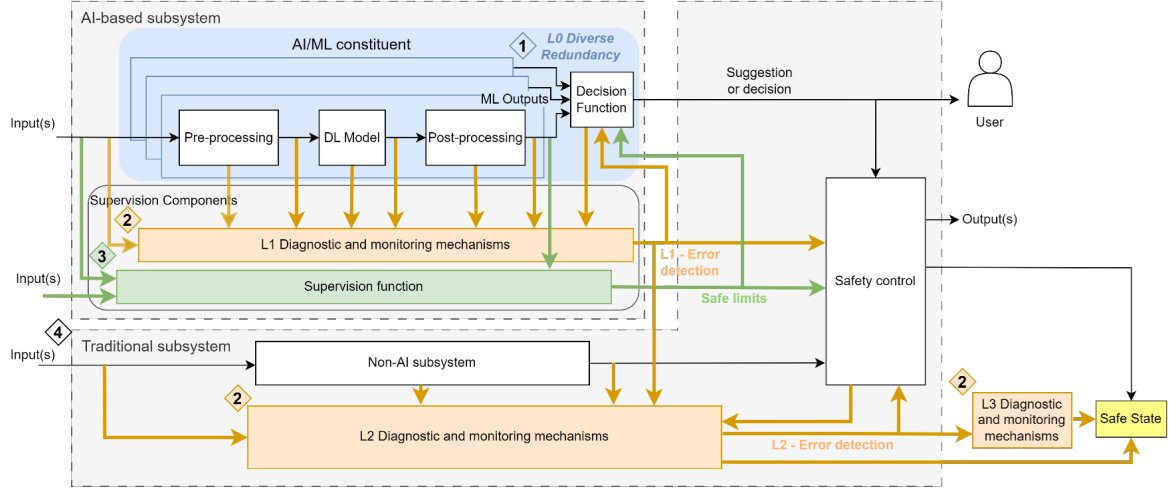


Fig. 3: Reference safety architecture pattern for safe AI-based systems

can implement diverse redundancy if required to meet the allocated safety requirements (e.g., ASIL-D in automotive systems). Traditional lockstep solutions can be employed. However, if the required hardware support for such lockstep is not available or higher accuracy and robustness is wanted, diverse redundancy based on input data transformations for the AI/ML model, devised in the context of SAFEXPLAIN, can be used [14], [15].

2) **Decision Function:** The AI/ML constituent implements a diverse redundancy scheme to enhance the performance of the DL model and increase the ratio of AI subsystem errors that can be detected and controlled. Within this diverse redundancy scheme, the Decision Function, takes the output from all the redundant nodes and provides a single prediction. Then, according to the DL usage level, the Decision Function might provide a suggestion (e.g., warning message, status information) and / or decision (e.g., command on the actuators) to the user and / or safety control. For instance, some decision functions explored in the context of SAFEXPLAIN to combine the outputs of diverse redundant AI/ML models include averaging their outputs, voting, or “joining” (non-maximum suppression) their predictions [14], [15].

3) **Supervision components:** These components consist of the L1 diagnostic and monitoring (L1DM) mechanisms as well as the supervision function. The former, comprises the lower-level diagnostics and monitoring in the platform, which complements the DL model diverse redundancy for detecting runtime errors or model insufficiencies and anomalies on the AI/ML constituent. To this end, the L1DM mechanisms gathers information from the AI/ML constituent (i.e., inputs, pre-processing, DL model, post-processing, decision function) and provides information on the detected errors to the decision function and L2 diagnostics and monitoring (L2DM) mechanism. The latter, bounds the AI/ML constituent operations to work within a predefined safety envelope. To this end, it collects information from the inputs/outputs of the AI/ML constituent and provides a set of constraints or limits to the decision function.

B. Traditional subsystem

This subsystem comprises the components usually present in a traditional (non-AI based) system.

1) **Non-AI subsystem:** Depending on the DL usage level, the non-AI subsystem can have different usage, it can either be the main safety element, or a fallback element that is activated whenever the supervisor identifies an unsafe operation of the AI-based subsystem.

2) **L2 diagnostic and monitoring mechanisms:** Comprises platform-level diagnostic and monitoring following traditional functional safety techniques. To this end, it gathers information from all the components within the traditional subsystem (i.e., inputs, non-AI subsystem, safety control) as well as the L1DM mechanism (i.e., L1 - Error detection) and provides information on the detected errors (L2 - Error detection) to the safety control and L3 diagnostics and monitoring (L3DM) mechanism. Considering the error information from lower-level diagnostic and monitoring mechanisms (i.e., L0 and L1), the L2DM is responsible of triggering the required reaction for error handling (e.g., take the system to a safe state).

3) **Safety control:** Collects the output information from the AI/ML constituent and the non-AI subsystem as well as safety related information from the diagnostic and monitoring components (i.e., L1DM, L2DM) and the supervision function (i.e., safe limits) to implement the system control and actuation logic (i.e., command system output(s) or take the system to a safe state). For instance, if the non-AI subsystem implements a fallback function for the ML constituent, in the event that a functional safety problem is detected in the AI-subsystem, the safety control might decide to discard the suggestion/decision from the decision function and operate according to the fallback function. The safety control might also limit the output(s) to meet with the safe limits defined in the supervision function or even take the system to a safe state.

C. L3 Diagnostic and monitoring mechanisms (L3DM)

Following traditional functional safety practices, some diagnostic may be implemented on an external device such as an external watchdog or microcontroller. Considering the error information from the L2DM mechanisms (i.e., L2 - Error detection), the L3DM mechanism is responsible of triggering the required reaction for error handling (e.g., take the system to a safe state).

IV. AI-SPECIFIC SUPERVISION COMPONENTS

While a number of elements in the safety architecture pattern have been deployed to enable the use of AI, those devoted to supervision are the most novel ones in the context of a safety architecture and are, in most cases, AI-specific. In particular, those components build on Explainable AI (XAI) principles.

XAI approaches are required to be applied at different phases of the AI-FSM lifecycle including: Data Management – to assess required distributions and quality of data; Learning Management – to assess whether the selected model is optimal and optimized for the specific task(s) within the ODD/Operational scenarios; Inference Management – to assess whether the performance of the model can generalize, within defined safe boundaries, to making accurate predictions for real world data.

While the literature on XAI is extremely abundant, it is not the purpose of this paper to survey it. Instead, we provide a very high-level categorization of the XAI solutions available, and later on some details on how they can be used for supervision purposes in the context of the AI-FSM.

A. XAI Background

XAI solutions can be applied on the training data and on the trained DL model.

1) *Data explainability*: XAI methods can be used to explain the data used in a DL development/deployment project. We refer to these methods as “data explainers” in this section. Data explainers can be used to provide insights into datasets and/or datapoints within a dataset, such as skew, missing values, significant features, relationships between data points in the dataset (e.g., their distribution within the data space), and the like. Data explainers can also be used to summarize data (e.g., Auto Encoders [16], [17]), reduce data dimensionality, and reveal dataset structure and feature importance.

2) *Model explainability*: Model explainability refers to the techniques to enhance explainability of DL models through various approaches that provide insights into how models make decisions, represent information, and process inputs to produce outputs. The following methods can be employed to achieve this:

- **Feature and Attribution Methods**: These techniques help understand the influence of different input features on the model’s predictions and decisions.
- **Model Structure Interpretation**: This involves analysing the architecture design of the DL model, such as its organization and connections within the model, and boundaries that separate different decision regions.
- **Semantic Understanding Methods**: These methods assess whether the internal decision-making process of the DL model aligns with domain concepts.
- **Interpretable Model Design**: This approach involves designing DL models with interpretability in mind, either by using model architectures that provide explicit explanations for their decisions, or by incorporating hook checkpoints to export explanations when required.
- **Rule Extraction Methods**: These techniques describe the model’s decision-making process in terms of rules, allowing domain experts to verify whether the rules align with their expertise.
- **Uncertainty and Reliability Analysis**: This involves quantifying the residual uncertainty of the model’s predictions, which is essential for mitigating uncertainty

during system design, and to control risk levels during inference.

- **Enhanced Representation Methods**: These techniques design DL models with mechanisms that encourage them to derive predictions based on approved reasoning, such as attention mechanisms, and custom layers and structures for interpretability.

B. Example of XAI Use for Supervision

1) *Uncertainty-aware models*: Although anticipated uncertainties have been mitigated throughout the various AI-FSM phases and steps, residual uncertainties may remain, resulting in need of real-time monitoring during operation to maintain an acceptable level of safety risk of encountering hazardous situations. Uncertainty-aware models refer to a set of design approaches that aim to enhance the target DL component(s) with extra capability to quantify the uncertainties associated with in-distribution data, as well as potential out-of-distribution scenarios. These techniques provide real-time estimates of uncertainties alongside the model’s predictions, thereby furnishing the decision-making module with the necessary additional information to make safe and informed decisions. Typical methods that can be used to provide uncertainty estimates include [18]: (1) Model epistemic uncertainty estimates (e.g., ensemble methods, Monte Carlo dropout, and Bayesian neural networks), and (2) aleatoric uncertainty estimates. An illustration of model epistemic uncertainty quantification using Monte Carlo (MC) dropout is shown in Figure 4.

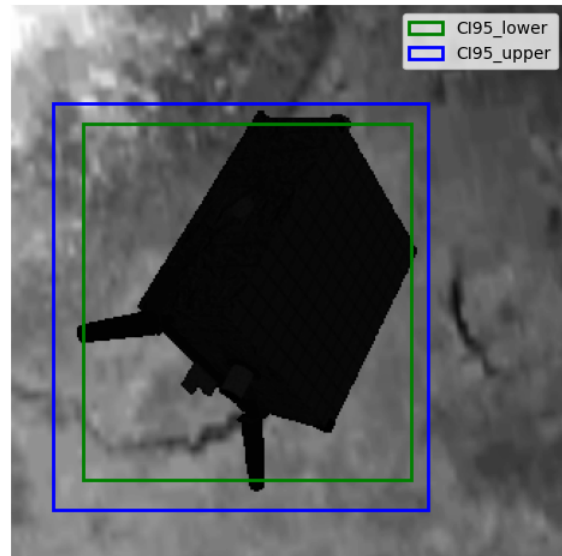


Fig. 4: Epistemic uncertainty quantification for detected object bounding boxes using SSDLite model as object detector with Monte Carlo (MC) dropout

2) *Out Of Distribution Detectors*: Out-Of-Distribution (OOD) detectors or anomaly detectors are used to identify data points that deviate from the established distribution of the datasets managed by AI-FSM Data Management. These detectors are required to ensure that the model safely operates within its certified safe boundaries and avoid the risk of getting into hazardous situations that result from the epistemic type of uncertainties, including domain uncertainties and

part of model epistemic uncertainty. Techniques that can be used as candidates for OOD detectors include: (1) statistical methods (e.g., distance-based, clustering, Gaussian mixture models, and kernel density estimation), and (2) Machine Learning based methods (e.g., Variational Auto Encoders, generative adversarial networks). An example of utilizing an Out-of-Distribution (OOD) detector to identify anomalous objects with novel appearances, never seen during training, is illustrated in Figure 5 (in relation to the example image used in Figure 4).

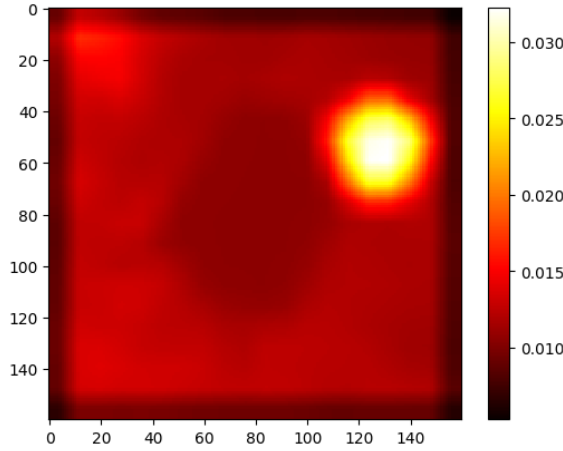


Fig. 5: VAE-based OOD detector identifies anomalous areas within the input image

We have also conceptualized XAI use for supervision in terms of disentangling sources of uncertainty [19]. In this case a source of uncertainty, in addition to model epistemic uncertainty and aleatoric uncertainty, that may induce unreliability in the deep learning model prediction, is domain uncertainty, which concerns the variability in the input data distribution [18].

3) *Surrogate models*: Safe surrogate models are simplified, interpretable models that learn to mimic the behaviour of the main DL models by being trained on the same input data (and the main model corresponding predictions). Although the surrogate model's predictions may not be as accurate, its internal logic is transparent and explainable, enabling safety experts to certify it within the AI-FSM development lifecycle. The predictions generated by the surrogate model are considered safe rule-based, where the rules have been pre-certified by experts within the AI-FSM lifecycle. When the main model's predictions diverge from those of the safe surrogate model, it indicates that the main model may be relying on unverified internal mechanisms to produce its outputs, compromising the required level of trustworthiness. During operation, if the discrepancy between the two models' predictions exceeds a predetermined acceptable limit, the main model's prediction will be rejected for use in safety-critical decisions.

C. Decision Function

The decision function aggregates outputs from both diverse DL components (as explained before) and other components within the supervision component container, including anomaly detectors, surrogate models and uncertainty aware models. It generates a consolidated prediction, together with

trustworthiness score and relevant explanations which are provided in the event of a rejection.

V. SAFEXPLAIN'S PLATFORM

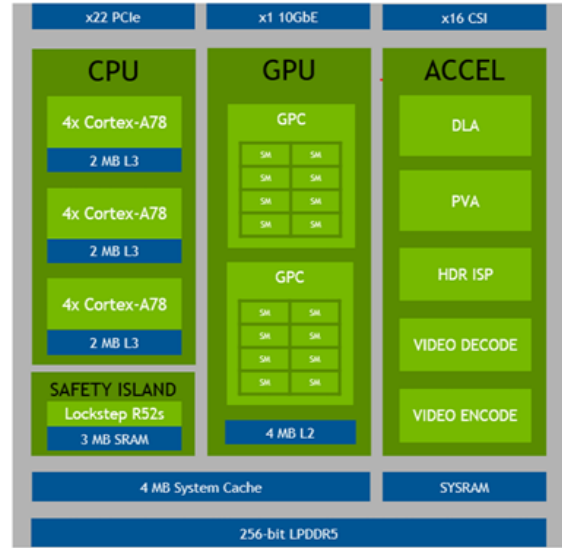


Fig. 6: Block Diagram of our target platform (from [20]).

A. Baseline Hardware and Software Stack

Based on the technological and performance requirements emerging from the use cases, the project partners reached a consensus on the adoption of the NVIDIA AGX Orin [20] as target platform. The NVIDIA Jetson AGX Orin is a family of heterogeneous MPSoC (Orin 32/64 Nano) developed by NVIDIA to cover the emerging requirements from diverse markets, all sharing the need for high-performance to support AI-based functionalities at reduced SWaP (Size, Weight, and Power). In SAFEXPLAIN, the AGX Orin Dev Kit has been selected. The Orin comprises 3 clusters of 4 Arm Cortex-A78AE CPUs, a NVIDIA Ampere GPU, ad-hoc AI-oriented accelerator such as NVDLA and PVA, as well as a video encoder and a video decoder (see Figure 6). The system also exploits a high-speed IO, with 204 GB/s of memory bandwidth, and 32GB of DRAM (in the Dev Kit version). The Orin can deliver up to 275 TOPS, which enable the execution of multiple concurrent AI applications.

The NVIDIA AGX Orin [21] comes with tailored OS support and libraries. The low-level software stack includes a tailored instance of an Ubuntu release also called Linux Tegra version. The set of tools normally available on Linux systems is complemented by the NVIDIA proprietary SDK JetPack, providing the necessary support to exploit the programming of the GPU and accelerators and the exploitation of standard deep learning libraries.

B. Platform- and System-level V&V Support

Our platform strategy builds around the concept of SAFEXPLAIN Middleware as a common ground for deploying and integrating all technological elements developed to support analysis and platform level concerns in general. Figure 7 provides a high-level view, on an architectural example, of the large and diverse set of platform-level elements and tools that are supported by the SAFEXPLAIN execution platform.

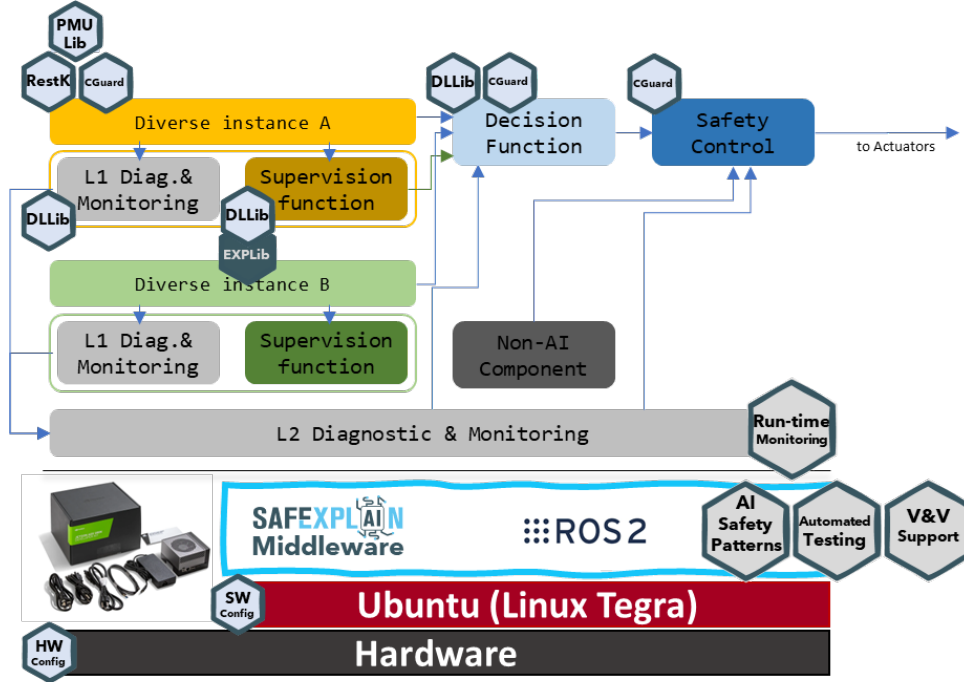


Fig. 7: Summary of platform-level support.

Those elements include non-AI specific supervision (e.g. PMULib and CGuard) and performance analysis (e.g. RestK in [1]). The interesting information we can draw from the picture is that the SAFEXPLAIN ecosystem includes a set of tools and methodologies that operate on the platform configuration at hardware and OS level and enable SAFEXPLAIN building blocks (Safety Patterns, Explainable AI, V&V support, Predictability, and Performance). The concept of Middleware is a critical enabler for the whole ecosystem.

The SAFEXPLAIN middleware has been designed to serve as a common abstraction layer to accommodate all platform level requirements on top of the execution platform (hardware and system software). This provides a consistent execution environment where all SAFEXPLAIN tools and methodologies are naturally integrated and can be straightforwardly exploited by the platform user at development, analysis, and operation.

The middleware has been designed to capture both self-imposed requirements and all sorts of requirements emerging from the different solutions in the project. It positions in between the execution platform and the user application. The Middleware has been instrumental to:

- Ensure compliance with AI-FMS and safety architectural patterns by design, avoiding the complexities and the pitfalls of delegating the full design to the end user, and ensuring the system is deployed following the selected safety pattern.
- Facilitate the integration of the use cases on top of a simplified ROS2-based [22], [23] framework where low-level configurations become transparent to the user, and the porting effort is often reduced to move code snippets within predefined placeholder nodes in the middleware.
- Provide automation support for recurrent V&V tasks, with special focus on providing an efficient environment for SAFEXPLAIN verification and validation require-

ments and tools.

- Seamlessly support traditional software-level monitoring solutions, by collecting functional and non-functional metrics at operation and support state-of-practice life-cycle monitoring and management.

The middleware has been designed and developed to support the execution of the use cases in a consistent environment where functional safety and explainable AI concepts are implemented. The middleware is also supporting the integration of DL and XAI libraries on top of the hardware and software stack and consistently with the functional safety architecture and the concrete Safety Patterns.

VI. CASE STUDIES

This section provides some brief information on the case studies currently under evaluation in SAFEXPLAIN where the devised technologies have been successfully integrated. Apart from those domain-specific case studies, the project has already released webinars describing the end-to-end design process and tools, as well as a domain-agnostic demo showcasing the use of SAFEXPLAIN technologies in real systems (see <https://safexplain.eu/demos-and-videos/>).

A. Railway Case Study

The SAFEXPLAIN railway case examines the viability of a safety architectural pattern for the completely autonomous operation of trains (Automatic Train Operation, ATO).

The project employs intelligent DL-based solutions, including artificial vision elements, to detect and locate people and obstacles on the track and in the way of the train doors and to estimate their position to ensure the train does not collide with obstacles or injure passengers. Safety-related software elements and DL software elements implement the safety function that allow trains to make safe and optimal decisions in real-time.

B. Space Case Study

Space missions are part of a leading sector in technology development and research. Using the state-of-the-art mission autonomy and AI technologies will enable fully autonomous operations during space missions and optimize mission costs. These technologies are tested using high safety-critical scenarios.

The current approach to space mission operations is laborious due to extensive use of ground operators to perform activities such as mission planning, telemetry monitoring, payload data analysis and failure mitigation.

C. Automotive Case Study

Rapid advancements in AI and autonomous vehicle technology, self-driving hold the potential to revolutionize transportation. In this context, SAFEXPLAIN works on designing detection systems that adhere to both safety and explainability requirements.

The SAFEXPLAIN automotive case study develops advanced methods and procedures that enable self-driving cars to accurately detect road users, estimate their distance from the vehicle, and predict their trajectories. These capabilities are vital for making timely decisions and executing appropriate collision avoidance maneuvers.

VII. CONCLUSIONS AND NEXT STEPS

The use of AI in safety-critical systems is disruptive due to the opposed nature of the safety-relevant development processes and the intrinsic characteristics of AI software. Therefore, there is a need for rethinking the development processes, the software architectures, and the platform support needed to reconcile AI software characteristics with the safety culture driving developments in safety-critical systems.

SAFEXPLAIN provides an end-to-end solution to those challenges. This paper has shown the main characteristics of the approach and how all pieces fit together. At the time of writing this paper, a demo case study is already complete and is in process of being packaged for its external sharing. Simultaneously, three case studies in the automotive, space and railway domains, already presented in the past [24], are being integrated on top of SAFEXPLAIN's technology. Results and experiences so far are very positive, and results for the use cases will see the light around September. These activities will allow complete the activities planned in the project. However, the (large) success of the project, which has brought a lot of attention from end users and practitioners, brings plenty of opportunities for commercial exploitation and further research, which will be planned and agreed during forthcoming months.

ACKNOWLEDGEMENTS

The research leading to these results has received funding from the Horizon Europe Programme under the SAFEXPLAIN Project (www.safexplain.eu), grant agreement num. 101069595.

REFERENCES

- [1] International Electrotechnical Commission, *IEC 61508. Functional Safety of Electrical/Electronic/Programmable Electronic Safety-related Systems*, 1998.
- [2] J. Perez-Cerrolaza, J. Abella, M. Borg, C. Donzella, J. Cerquides, F. J. Cazorla, C. Englund, M. Tauber, G. Nikolakopoulos, and J. L. Flores, "Artificial intelligence for safety-critical systems in industrial and transportation domains: A survey," *ACM Comput. Surv.*, vol. 56, no. 7, Apr. 2024. [Online]. Available: <https://doi.org/10.1145/3626314>
- [3] A. Brando, I. Serra, E. Mezzetti, F. J. Cazorla, J. Perez-Cerrolaza, and J. Abella, "On neural networks redundancy and diversity for their use in safety-critical systems," *Computer*, vol. 56, no. 5, pp. 41–50, 2023.
- [4] International Organization for Standardization (ISO), *ISO/IEC DTR 5469 Artificial intelligence - Functional safety and AI systems*, 2023.
- [5] "ISO/CD PAS 8800 Road Vehicles — Safety and artificial intelligence," 2023.
- [6] International Standards Organization, *ISO/DIS 26262. Road Vehicles — Functional Safety*, 2009.
- [7] "ISO/IEC WD 5338 Information technology — Artificial intelligence — AI system life cycle processes," 2023.
- [8] European Union Aviation Safety Agency (EASA), "EASA Concept Paper: guidance for Level 1 & 2 machine learning applications," 2023.
- [9] R. Hawkins *et al.*, "Guidance on the Assurance of Machine Learning in Autonomous Systems (AMLAS)," 2021.
- [10] "Automotive SPICE® Process Assessment / Reference Model Version 4.0," 2023.
- [11] J. Fernández *et al.*, "AI-FSM," 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.10964402>
- [12] J. Fernández, I. Agirre, J. Perez-Cerrolaza, L. Belategi, A. Adell, C. Donzella, and J. Abella, "AI-FSM: Towards Functional Safety Management for Artificial Intelligence-based Critical Systems," in *CARS@EDCC2024 Workshop - Critical Automotive applications: Robustness & Safety*, Leuven, Belgium, Apr. 2024. [Online]. Available: <https://hal.science/hal-04769949>
- [13] E. W. Group, "Standardized E-Gas monitoring concept for engine management systems of gasoline and diesel engines. version 4.0," https://downloads.regulations.gov/NHTSA-2014-0108-0021/attachment_1.pdf, 2007.
- [14] M. Caro, A. Brando, and J. Abella, "Semantic diverse dmr and tmr for high-integrity ai-based function efficiency," *ACM Trans. Cyber-Phys. Syst.*, vol. 9, no. 2, Mar. 2025. [Online]. Available: <https://doi.org/10.1145/3716140>
- [15] —, "Software-Only Semantic Diverse Redundancy for High-Integrity AI-Based Functionalities," in *European Real-Time Symposium (ERTS)*, Toulouse, France, Jun. 2024. [Online]. Available: <https://hal.science/hal-04614881>
- [16] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2022. [Online]. Available: <https://arxiv.org/abs/1312.6114>
- [17] A. Kumar, P. Sattigeri, and A. Balakrishnan, "Variational inference of disentangled latent concepts from unlabeled observations," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. [Online]. Available: <https://openreview.net/forum?id=H1kG7GZAW>
- [18] A. Brando, I. Serra, E. Mezzetti, F. J. Cazorla, and J. Abella, "Standardizing the probabilistic sources of uncertainty for the sake of safety deep learning," in *SafeAI@AAAI*, 2023. [Online]. Available: <https://ceur-ws.org/Vol-3381/11.pdf>
- [19] R. Lowe, M. Ulan, T. Bui, A. Adell, J. Labaien, and A. Brando, "Managing sources of uncertainty in utilizing ai in development and deployment in safety-critical autonomous systems," in *IEEE 22nd International Conference on Software Architecture (ICSA 2025)*, 2025.
- [20] NVIDIA, "NVIDIA Jetson AGX Orin Series - A Giant Leap Forward for Robotics and Edge AI Applications - Technical Brief," 2022.
- [21] —, "NVIDIA Jetson AGX Orin Developer Kit User Guide," <https://developer.nvidia.com/embedded/learn/jetson-agx-orin-devkit-user-guide/index.html>, 2023.
- [22] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022. [Online]. Available: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>
- [23] S. Macenski, A. Soragna, M. Carroll, and Z. Ge, "Impact of ros 2 node composition in robotic systems," *IEEE Robotics and Autonomous Letters (RA-L)*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.09933>
- [24] J. Abella, J. Perez, C. Englund, B. Zonooz, G. Giordana, C. Donzella, F. J. Cazorla, E. Mezzetti, I. Serra, A. Brando, I. Agirre, F. Eizaguirre, T. H. Bui, E. Arani, F. Sarfraz, A. Balasubramanian, A. Badar, I. Bloise, L. Feruglio, I. Cinelli, D. Brighenti, and D. Cunial, "Safexplain: Safe and explainable critical embedded systems based on ai," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2023, pp. 1–6.